

Web Markup and Role of Documents

How the Web Became an App Platform and Lost Its Document Roots

spiritofstar

October 2026

0.1 Abstract

This article explores how the web's shift toward an app-driven economy sidelined digital reading experience. It examines how modern browsers are slowly trying to catch up with better typography tools, why file formats like PDFs still dominate despite their flaws, and what it would take to rebuild the web as a hub for digital publishing and long-form reading.

0.2 World Wide Web

World Wide Web from its inception started as a Scientific Document Sharing Network, which slowly became more interactive as Popularity and usage of the Web grew, currently, Webpages (particularly HTML + CSS pages) are focused on interactivity, such as Scroll-based Animations, Keyframes and rarely on being Document, as a result of the Web pivoting to being an App Platform rather than Document platform, to be honest, i am fine with that, but one particularly pressing and persistent problem is that Web Proposals that could enhance Reading experience and Typesetting is usually put aside in favor of App-oriented proposals, this particularly applies to CSS, which originally was supposed to serve purpose of styling documents, but now it somehow feels like HTML+CSS are Declarative UI Programming and JavaScript is supposed to handle app logic, which is weird turn, because we are losing quality of Document-sharing that Web had, and i strongly believe that pivot is why Knowledge-sharing quality of Internet we know and love is deprioritized in favor of quick dopamine-hits of Mass Social Interaction, Advertising Revenue, Surveillance Capitalism.

0.3 Reading Experience & Publishing Industries

Web was such a good model for Digital Books and Documents that entire formats were based off of XML, XHTML/HTML, as a result, EPUB, XML Paper Specification, Office Open Formats, OpenDocument Formats emerged to list notable few, CSS was used to enhance and style Markup Documents and Markup-based Formats, such as HTML and EPUB, eventually, CSS slowly but drastically changed to accommodate App boom that Web was undergoing, HTML and CSS never fully got Document Typesetting features that we would have gotten earlier if it wasn't for Concept of App taking over the Web(namely CSS Books), **CSS Books** (formally hosted as a WHATWG specification draft, *CSS Books: Living Standard*) was an ambitious attempt to consolidate and standardize high-end typography and book-publishing mechanics into a single web specification. However, with time some of similar concepts made into newer specifications, some are Drafts, some aren't adopted, because of Browser politics and the fact that Web Browsers cater to most profitable industries, We are slowly getting those CSS Proposals that would allow for Typesetting and Typographic controls, but adoption/consistent support is slow, even though it is indeed progressing, CSS Paged Media, Generated Content for Paged Media, Fragmentation, Text Modules(with varying levels and features) are mostly being adopted by Browsers, Safari pioneered hanging-punctuation and boasts the most typographically accurate implementation

of text-wrap: pretty, evaluating paragraph breaks holistically rather than using behavioral heuristics for performance, hence why Safari's implementation of text-wrap-styles as well as their early adoption of hanging-punctuation makes them Pioneering Browser for Typesetting Workflows for many of the Web tinkerers Moreover, Web is not that relevant in Document-related industries, where PDF is insanely dominant, and Office/OpenDocument and EPUB formats serve some role too, though not nearly as ubiquitous as PDF, and since PDFs are most distributed Document on Web as well(ironically) Browsers are prompted to support it natively, even over formats such as EPUB(which is Standard Specification defined by W3C organization that also defines World Wide Web related standards) and even though it makes more sense to support EPUB from engineering perspective, given that it is ZIP bundle of XHTML, CSS, and Assets. Browsers have to write PDF Readers from Scratch that historically generated significant attack surface (memory corruption bugs, zero-day vulnerabilities). Despite this security cost, PDF support remains non-negotiable for browsers because it is the global standard for business and legal documents.

0.4 Modern Markup Languages

As browser engines focused on application runtimes and WYSIWYG(Visual) HTML Editor faded because of complexity, content creators and technical authors began seeking simpler, lightweight alternatives to both complex HTML/CSS abstractions and proprietary word processors. This reaction sparked a major shift toward plain-text markup languages.

0.4.1 Markdown and the Web of Plain Text

Markdown succeeded because it stripped away visual formatting bloat and focused strictly on human readability and ease of adoption. By compiling clean text directly into semantic HTML (# mapping to <h1>, ** to), Markdown preserves structural hierarchy, making documents easily indexable, archivable, and accessible to assistive software. The explosive growth of static site generators and digital gardens demonstrates a enduring demand for simple, document-first web publishing.

0.4.2 Typst: Academic & Book Typesetting

While our loved and adored Markdown handles basic web notes, complex academic and scientific publishing historically relied on **LaTeX** a typesetting system known for slow compilation speeds, obscure syntax, and painful setup environments. modern culmination of Markup, Styling, Typesetting, Document Creation is all in Typst, a modern Typesetting System, as well as markup and scripting language, which has easy Markdown-like syntax and can export to Semantic HTML and PDF, compile in milliseconds rather than seconds, Typst provides a real-time preview of book-grade typesetting, baseline grids, and mathematical formulas. It serves as a practical blueprint for how a native, document-centric Web could have been, especially observable parallels between CSS(the Stylesheet language) and Typst.

0.5 Semantic Web

The Semantic Web builds upon the foundation of web to make online information understandable not just to people, but to machines, namely Assistive Software for people with disabilities. While the early web revolutionized how we share documents through **hypertext** connecting web pages via hyperlinks, it relied heavily on humans to read and interpret the meaning of that content. A traditional link might take you from a page about an author to a page about a book, but a web browser has no inherent understanding of the *relationship* between them. The Semantic Web extends this hypertext paradigm by giving structure and explicit meaning to data. By using standardized frameworks like RDF (Resource Description Framework) and ontologies, it transforms the web from a collection of linked text documents into a unified, machine-readable web of data, which formed Knowledge Graphs, made content Accessible, pioneered Digital Gardens all based on foundational belief in the power of context-rich, meaningful connections across human knowledge.

PDF effectively serves as an antithesis of that, Inside a PDF, words are stripped of their semantic context and converted into raw geometric instructions, because the format lacks a true, semantic tree by default, screen readers and assistive software struggle to parse reading order, navigate multi-column layouts, or read tables accurately. Optimal application of the PDFs is to be printed but it is unfortunately often used outside of it's primary purpose because of the broad support it enjoys.

0.6 Security, Standards and Society...

Internet is victim to many exploits historically, PDF isn't only one of them, JavaScript, Image Codecs, Font Parsers, WebGL/WebGPU introduce attack surfaces that increase potential for exploitation, typically Standards Bodies, Browsers are supposed to consider that and limit what specific introduced standard can do, Moreover, at the beginning of the Web, cosmetic additions and Plugins were exceptionally common for Browsers, Plugins allowed Binary Code to be added to Browser and each Browser had Plugin Interfaces for that reason, IE had ActiveX Plugins, Firefox had Netscape Plugin API(NPAPI), Chrome had Pepper Plugin API/NaCL(PPAPI), Safari had NPAPI as well, Web Community often popularized insecure Plugins for sake of interactive multimedia, namely the Flash Player, which at one point started being shipped by default, Flash Player gave Rich Interactive Content that Web initially lacked, Particularly, Vector Animations(SWF), Flash Movie Format, Scripting. basically to summarize Flash was Application Runtime within Web before Web itself became one with it's attempts to replace it, one of the reasons why Flash died out was fact that Steve Jobs disliked it and refused to include/support it on iOS, causing Web and App developers to slowly pivot from it, Steve Jobs also released a open letter titled "Thoughts on Flash", which immensely criticized it mainly based on the fact that Flash was not Open Standard, it was insecure closed source Proprietary Runtime that was dependent on Adobe's Decisions(ironically, same criticism was relevant and partially still it regarding PDFs), after that fact, HTML5 standard eventually released and Browsers adopted it, Flash continued to decline as Native Browser technologies surpassed it and Browsers eventually removed support for it. PDF and SWF were standardized around the same time, part of the reason why PDF survives on Web today comes down to **utility versus replaceability** Flash was an interactive app runtime built on closed, proprietary code. Once open web tools like HTML5 and CSS grew powerful enough to build games, animations, and video players natively, Flash lost its main purpose. Modern browsers could easily replace Flash's functionality without taking on its severe security baggage.

PDFs, on the other hand, filled a non-negotiable need that the web failed to solve: **predictable, unchangeable documents**. Businesses, courts, and governments required documents that looked identical on every device, print-ready, and legally binding. Because web standards never fully matured to handle high-end document formatting, society stayed locked into PDF forcing web browsers to build complex, built-in PDF readers despite the recurring security risks they bring.

0.7 Guided Trajectory for future of the Web

The web's evolution over the past decade prioritized application performance, dynamic execution, and interactive experiences, often treating long-form content and document layouts as secondary concerns. The failure of initiatives like **W3C Web Publications** and the **WHATWG CSS Books** efforts to gain adoption left long-form reading on the web in an fragmented state. However, there is still progress on the Web for Text-based Media, like previously discussed CSS Paged Media and Text Modules that need more traction, and they remain main purpose of this article, Popularizing Readable, archivable, accessible Web is net-positive for everyone in Age of Information.